



NestJS Course

Day [1]



Agenda

- **Introduction**
- **Installation**
- **NestJS structure**
- **Controllers**
- **Services**
- **Modules**
- **Exception filters**
- **Middlewares**



Prerequisites

- Javascript
- Typescript
- NodeJS
- HTTP
- RESTful APIs
- Express



Introduction

What is NestJS?

NestJS is a **Node.js framework** for building server-side applications. It uses TypeScript by default, providing strong typing and modern Javascript features.

NestJS is designed to help developers create highly **scalable**, **maintainable**, and **testable** applications.



Why NestJS?

- The Lack of Structure in Node.js (Non Opinionated)
- Full TypeScript Support.
- Familiarity for Developers from Other Ecosystems. (.NET Core, Java Spring Boot)
- The Rise of Microservices and Complex Systems.



History of NestJS

NestJS was created by Kamil Myśliwiec and was first released in December 2017. [Github repo](#)

It quickly gained popularity due to its robust features, TypeScript support, and modular architecture.

NestJS has since become a key player in the Node.js ecosystem, offering a structure that **accelerates development** and improves **code maintainability**.



Features of NestJS

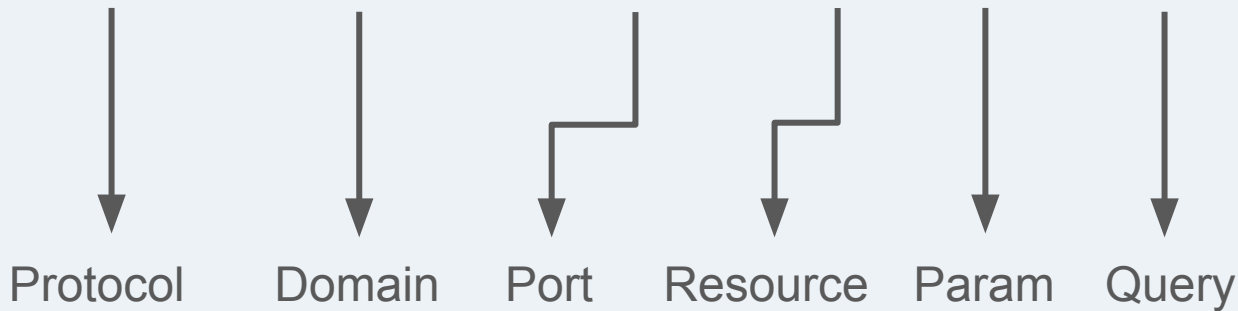
- **Modular Architecture:** Provides a modular structure, adds reusability and maintenance.
- **TypeScript by Default:** Provides type-checking and modern JavaScript features.
- **Dependency Injection:** Implements **DI** for managing dependencies.
- **Decorators:** Uses decorators for defining routes, middleware, making the code clean.
- **GraphQL:** support for GraphQL through dedicated modules.
- **Microservices:** Built-in support for building microservices architectures.





RESTful endpoint

<http://localhost:3000/users/1?dept=HR>





Dependency Injection

high-level modules should not depend on low-level modules.

both should depend on abstractions.



Without Dependency Injection

The class creates and uses the concrete EmailService directly.

```
1  // Concrete service (tightly coupled)
2  class EmailService {
3      async send(to: string, subject: string, message: string): Promise<void> {
4          // Imagine real email sending logic here
5          console.log('✉ Email sent to ${to}');
6      }
7  }
8
9  // Class that uses EmailService directly (no DI)
10 class UserService {
11     private emailService: EmailService;
12
13     constructor() {
14         // Direct instantiation creates a hard dependency
15         this.emailService = new EmailService();
16     }
17
18     async register(userEmail: string, userName: string): Promise<void> {
19         // Business logic for user registration would go here
20         const subject = 'Welcome to Our Platform!';
21         const message = `Hi ${userName},\n\nThank you for registering with us!`;
22         await this.emailService.send(userEmail, subject, message);
23     }
24 }
```

✖ Drawbacks



Tight Coupling

UserService is tightly coupled to EmailService. Changes in EmailService can impact UserService.



Hard to Test

You cannot easily mock EmailService in tests. Unit testing becomes difficult and may require sending real emails or complex workarounds.



Not Flexible

Cannot easily switch to another notification channel (e.g., SMS, Push) without modifying UserService code.



Violates SOLID Principles

Specifically violates the Dependency Inversion Principle (DIP) — high-level modules depend on low-level modules.



Harder to Maintain

Direct dependencies spread across the codebase make the system harder to evolve and maintain.



Dependency Injection with Notification Service

// 1. Define the contract (interface)

INotificationService.ts

```
export interface INotificationService {  
  send(to: string, subject: string, message: string): Promise<void>;  
}
```

// 2. Create an abstract email service (implementation)

EmailService.ts

```
import { INotificationService } from './INotificationService';  
  
export class EmailService implements INotificationService {  
  async send(to: string, subject: string, message: string): Promise<void> {  
    // Here you would integrate with a real email provider  
    // (e.g., SendGrid, SES, Mailgun, etc.)  
    console.log(`✉ Email sent to ${to}`);  
    console.log(`Subject: ${subject}`);  
    console.log(`Message: ${message}`);  
  }  
}
```

// 3. A class that depends on the abstraction (interface)

UserService.ts

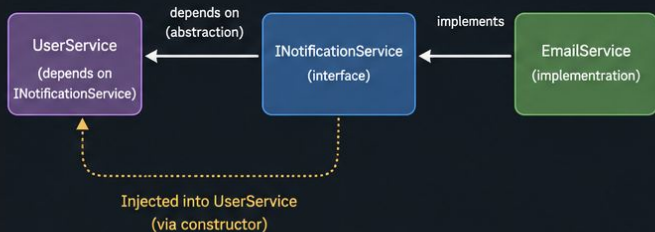
```
import { INotificationService } from './INotificationService';  
  
export class UserService {  
  private notificationService: INotificationService;  
  
  constructor(notificationService: INotificationService) {  
    this.notificationService = notificationService;  
  }  
  
  async register(userEmail: string, userName: string): Promise<void> {  
    // Business logic for user registration would go here  
    const subject = 'Welcome to Our Platform!';  
    const message = `Hi ${userName},\n\nThank you for registering with us!`;  
    await this.notificationService.send(userEmail, subject, message);  
  }  
}
```

// 4. Composition root - wire up dependencies

index.ts

```
import { EmailService } from './EmailService';  
import { UserService } from './UserService';  
import { INotificationService } from './INotificationService';  
  
// Create the concrete implementation  
const emailService: INotificationService = new EmailService();  
  
// Inject the dependency into UserService  
const userService = new UserService(emailService);  
  
// Use the service  
(async () => {  
  await userService.register('john.doe@example.com', 'John Doe');  
})();
```

How Dependency Injection works





Installation



Installation

```
npm i -g @nestjs/cli
```

```
nest new [project-name]
```

Note: You must install npm and node before running these commands



Structure



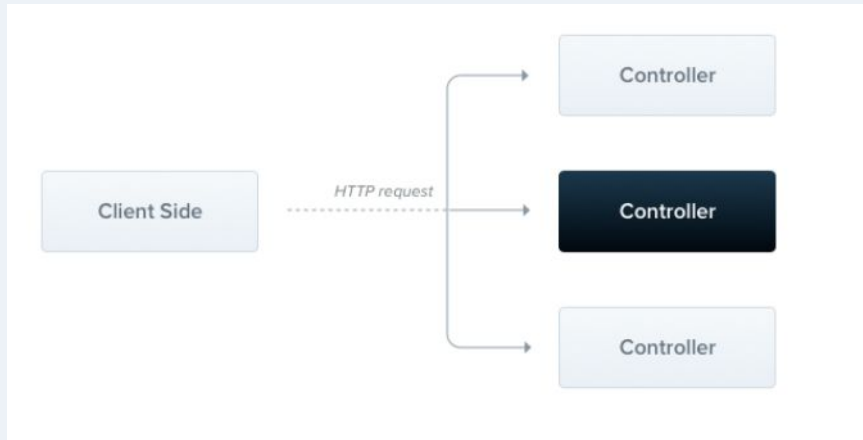
Controllers



Controllers

It's a class annotated with `@Controller()` decorator.

Controllers are responsible for handling incoming requests and sending responses to the client.



The **routing** mechanism determines which controller will handle each request.



Providers



Providers

A provider in NestJS is like a **worker** in a factory. It's responsible for doing a specific task (fetching data, processing requests).

A provider is a class that is annotated with the **@Injectable()** decorator. It can be **a service, helper, etc.**

Follows Singleton (by default) Only **one instance** exists per application.

The provider adds **reusability** to our system.



Modules



Modules

A module is a class that is annotated with the `@Module()` decorator. This decorator provides metadata that Nest uses to organize and manage the application structure efficiently.

Every Nest application has at least one module, the **root module**, which serves as the starting point for Nest to build the **application graph**.

This graph is an internal structure that Nest uses to resolve relationships and dependencies between modules and providers.



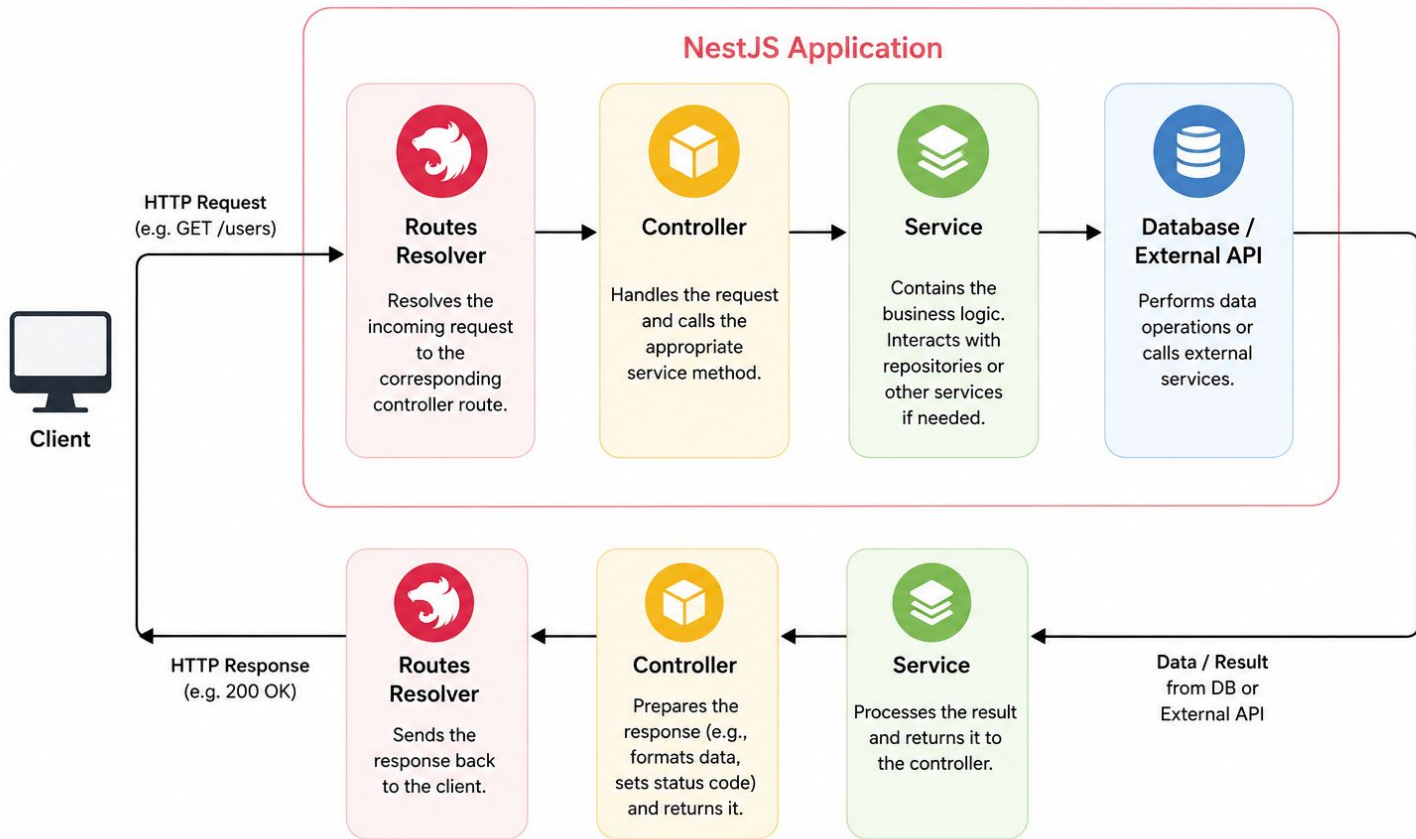


- **Imports** : Array of modules that the module depends on
Ex: Order Module depends on Payment Module
- **Controllers** : Array of controllers that handle different routes
Ex: user routes might have separate controller for profile, payments, etc.
- **Providers** : Array of services, helpers that is annotated with **@Injectable** decorator that holds business logic
- **Exports** : Array of services that the module can share with other modules





NestJS Request/Response Cycle





Implementation

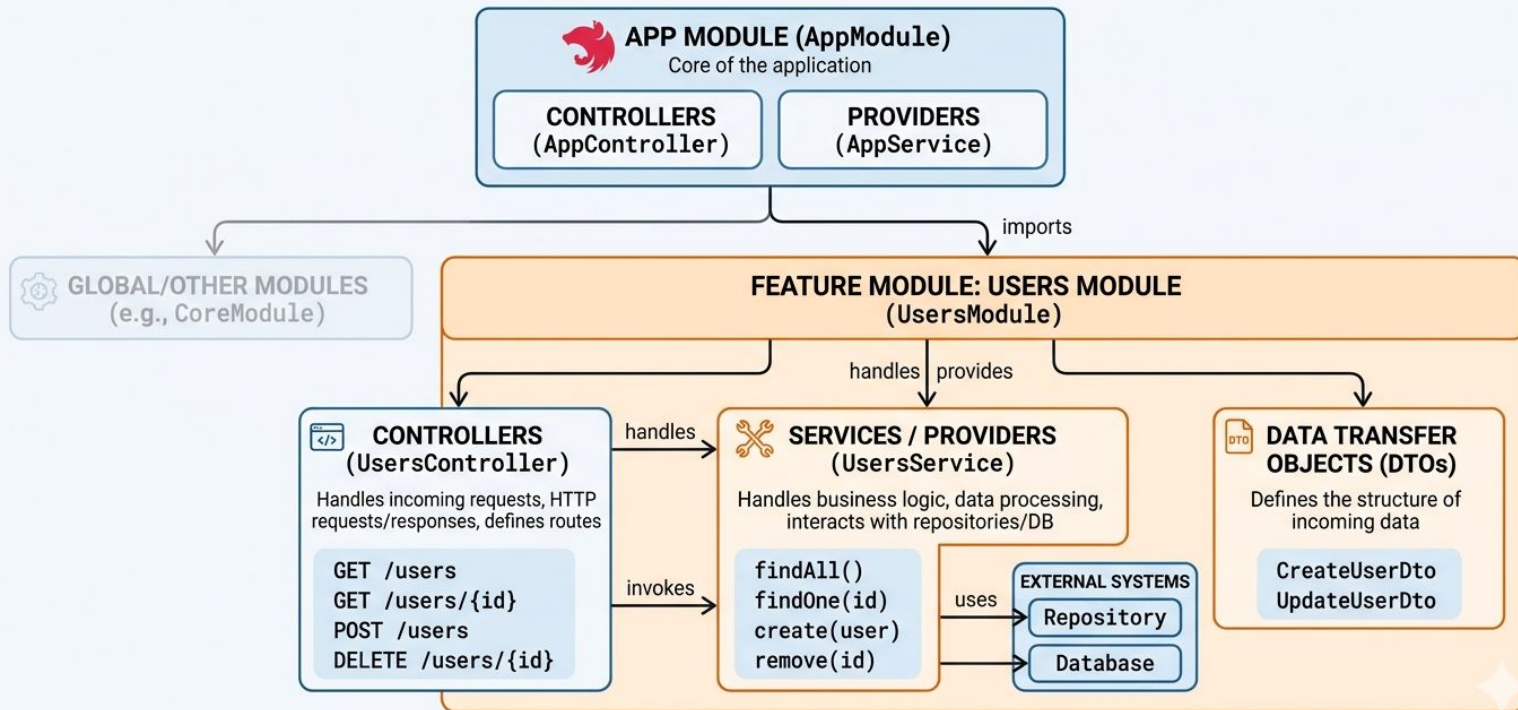


Create User Module

- Create It's CRUD endpoints
 - **GET** /users → Return all users
 - **POST** /users → Create user
 - **GET** /users/:id → Return user by id
 - **PATCH** /users/:id → Update user [Lab]
 - **DELETE** /users/:id → Delete user [Lab]



NESTJS APPLICATION HIERARCHY





IoC Container



Dependency Injection in NestJS

How the IoC Container works

1. Developer defines Providers (Modules, Providers, Consumers)

notification.service.ts

```
@Injectable()
export class EmailService
  implements NotificationService {
  send(to: string, message: string) {
    // send email
  }
}
```

user.service.ts

```
@Injectable()
export class UserService {
  constructor(
    private notificationService: NotificationService,
  ) {}

  register(user: CreateUserDto) {
    // ...
    this.notificationService.send(
      user.email,
      'Welcome to our platform!',
    );
  }
}
```

app.module.ts

```
@Module({
  providers: [
    { provide: NotificationService, useClass: EmailService },
    UserService,
  ],
  exports: [UserService],
})
export class AppModule {}
```

Legend

● Class / Provider
○ Token (Interface / Abstract / Symbol)
userService

→ Dependency
---> Provided by IoC Container



NestJS IoC Container

(Inversion of Control Container)



Step 1: Module Loading

Nest loads the AppModule and reads the providers metadata. It registers all providers in the container.



Step 2: Dependency Resolution

When UserService is requested, the container inspects its constructor and sees it needs:

- NotificationService



Step 3: Provider Lookup

The container looks for a provider that matches the token NotificationService. It finds: EmailService (useClass).



Step 4: Instance Creation & Injection

The container creates an instance of EmailService (if not already created) and injects it into UserService.



Step 5: Ready to Use

UserService is now fully constructed with its dependencies and returned by the container.

2. Container creates and wires (Runtime)

NotificationService
(Token / Interface)

provided by

EmailService
(Provider)

injected into

UserService
(Consumer)

★ Result

- ✓ UserService does not create EmailService.
- ✓ The IoC container provides it.
- ✓ Loose coupling
- ✓ Easy to test (mocks can be provided)
- ✓ Easy to swap implementations
- ✓ Follows Inversion of Control (IoC) principle

How it works (in order)

1

App starts
Nest creates the IoC container.



2

Module scan
Nest scans modules and registers providers.



3

Dependency request
A provider (e.g., UserService) is requested.



4

Resolve dependencies
Container resolves constructor dependencies recursively.



5

Instantiate & inject
Instances are created and dependencies injected.



6

Ready
Fully constructed provider is returned and cached.



Exception Filters



Exception Filters

Nest comes with a built-in exceptions layer, which is responsible for processing all unhandled exceptions across an application.

When an exception is not handled by your application code, it is caught by this layer, which then automatically sends an appropriate user-friendly response.

Nest comes with built-in exceptions such as **BadRequestException**, **NotFoundException**, **UnauthorizedException**, etc. Check this [link](#) for more types



Middleware



Middleware

Middleware is a function that is called before the route handler. Middleware functions have access to the **request** and **response** objects and the **next()** middleware.

What does middleware do?

- Execute any code. (ex: logging)
- Make changes to the req and res objects.
- End the request-response cycle.

Note: You **must** call **next()** to pass control to the next middleware function. Otherwise, the request will be left hanging.



Lab [1]

- Create a new Nest project.
- Create a Todo module with its CRUD operations.
- The todo object should have :
 - id : number
 - task : string
 - status: ENUM('todo', 'in-progress', 'done')
- Add middleware that logs the body and timestamp of the CreateTodo POST request only. [[hint](#)]
Ex: 2025-05-03T05:15:43.582Z { body }



Lab [1]

Endpoints:

- [GET] /todo
- [GET] /todo/:id
- [POST] /todo
- [PATCH] /todo/:id
- [DELETE] /todo/:id



Resources

- [What is NestJS?](#)
- [NestJs Documentation](#)
- [NestJS Dependency Injection](#)
- [Dependency Injection Article](#)
- [What is Inversion of control?](#)

